Pay-in Payout Module

A Laravel-based Pay-in / Payout management system with merchant wallet balances, API-key authentication, and a scheduled job for processing pending transactions.

Table of Contents

Requirements

- PHP >= 8.1
- Composer
- MySQL >= 5.7
- Laravel CLI (via `php artisan`)

Installation

Clone the repository and install PHP dependencies:

```
git clone https://github.com/ParasMajethiya1/Practical.git
cd Practical
composer install
```

Environment Configuration

Copy the example environment file:

```
cp .env.example .env
```

Open the newly created `.env` file and set the following values:

```
APP_NAME="Pay-in Payout Module"
APP_ENV=local
APP_KEY=
APP_DEBUG=true
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=payin_payout
DB_USERNAME=root
DB_PASSWORD=
```

Variable	Description
`APP_NAME`	Display name of the application
`APP_ENV`	Application environment (`local`, `production`, etc.)
`APP_KEY`	Encryption key, auto-filled by `php artisan key:generate`
`APP_DEBUG`	Enables detailed error output when `true`
`APP_URL`	Base URL used by the app and artisan serve
`DB_CONNECTION`	Database driver (`mysql`)
`DB_HOST`	Database host address
`DB_PORT`	Database port (default `3306`)
`DB_DATABASE`	Database name to use/create
`DB_USERNAME`	Database username
`DB_PASSWORD`	Database password (leave blank if none)

Generate the application key:

```
php artisan key:generate
```

Database Setup

Run migrations with seeders:

```
php artisan migrate --seed
```

If the database doesn't exist yet, you'll see:

```
The database 'payin_payout' does not exist on the 'mysql' connection. Would you
like to create it? (yes/no) [no]
```

Type `yes` to let Laravel create it automatically.

Once migration and seeding finish, credentials will be printed in the console — **save these immediately**, as they are not shown again:

```
Admin created | email: admin@example.com | password: ;#~T2Th.ttPY4n
  Database\Seeders\AdminSeeder
.....................................................................
... 564 ms DONE

  Database\Seeders\MerchantSeeder
.....................................................................
.... RUNNING
Merchant: Acme Retail Pvt Ltd | api_key: zo4FEMBc9bvXzXCMhD9OcJjKn9P6PUWvnWIpjQrE
| wallet balance: 50000.00
Merchant: Bright Traders | api_key: kwlbNRzSxSl27A3ecB8Z72EQI7NVfOz70hyBWn81 |
wallet balance: 10000.00
```

Item	Purpose
Admin email / password	Login credentials for the admin panel
Merchant `api_key`	Required in the `X-API-KEY` header for all merchant API requests
Wallet balance	Starting seeded balance for that merchant, used for payout validation

---

Running the Application

Start the local development server:

```
php artisan serve
```

The app will be available at `http://localhost:8000`.

Running the Cron / Scheduler

Pending payments (Pay-ins / Payouts) are processed by a scheduled artisan command.

For local, one-off testing, run the processing command manually whenever you want pending transactions processed:

```
php artisan payments:process-pending
```

For continuous processing that mirrors production behavior, run Laravel's scheduler loop in a separate terminal window (leave it running):

```
php artisan schedule:work
```

In production, do not use `schedule:work`. Instead, add a single cron entry on the server that triggers Laravel's scheduler every minute:

```
* * * * * cd /path/to/payin-payout-app && php artisan schedule:run >> /dev/null
2>&1
```

API Documentation

Base URL & Authentication

```
Base URL: http://localhost:8000/api/v1
```

All routes require the merchant's API key in the request header, **except** merchant registration:

```
X-API-KEY: <merchant_api_key>
```

---

1. Register a Merchant (Public)

Creates a new merchant account and returns an `api_key`. No authentication header required for this endpoint.

Endpoint: `POST /api/v1/merchants`

Body Parameters:

Parameter	Type	Required	Description
`name`	string	Yes	Full/legal name of the merchant business

Parameter	Type	Required	Description
`email`	string	Yes	Merchant contact email (must be unique)
`phone`	string	Yes	Merchant contact phone number

cURL Request:

```
curl -X POST http://localhost:8000/api/v1/merchants \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Acme Retail Pvt Ltd",
    "email": "acme@example.com",
    "phone": "9999999999"
  }'
```

Response:

```
{
  "success": true,
  "message": "Merchant registered successfully. Store the api_key securely - it
will not be shown again.",
  "data": {
    "id": 3,
    "name": "Acme Retail Pvt Ltd",
    "email": "acme@example.com",
    "api_key": "6QowJ...redacted...aB"
  }
}
```

> ⚠️ **Important:** The `api_key` is shown only once. Store it securely — you'll need it in the `X-API-KEY` header for every other request.

---

2. Initiate a Pay-in

Creates a new incoming payment transaction for the merchant.

Endpoint: `POST /api/v1/payins`

Headers:

Header	Required	Description
`X-API-KEY`	Yes	Merchant's API key
`Content-Type`	Yes	`application/json`

Body Parameters:

Parameter	Type	Required	Description
`amount`	numeric	Yes	Transaction amount (e.g. `1500.50`)
`currency`	string	Yes	Currency code (e.g. `INR`)
`payment_method`	string	Yes	Method used, e.g. `UPI`, `CARD`, `NETBANKING`
`customer_name`	string	Yes	Name of the paying customer
`customer_email`	string	Yes	Customer's email address
`customer_phone`	string	Yes	Customer's phone number
`remarks`	string	No	Optional note/reference, e.g. order ID

cURL Request:

```
curl -X POST http://localhost:8000/api/v1/payins \
  -H "X-API-KEY: <merchant_api_key>" \
  -H "Content-Type: application/json" \
  -d '{
    "amount": 1500.50,
    "currency": "INR",
    "payment_method": "UPI",
    "customer_name": "Rahul Sharma",
    "customer_email": "rahul@example.com",
    "customer_phone": "9876543210",
    "remarks": "Order #INV-1042"
  }'
```

Response:

```
{
  "success": true,
  "message": "Pay-in initiated successfully.",
  "data": {
    "transaction_id": "PIN20260902AB12CD34EF",
    "status": "PENDING",
    "amount": "1500.50",
    "currency": "INR",
    "created_at": "2026-09-02T10:15:00.000000Z"
  }
}
```

The transaction starts as `PENDING` and is moved to a final state (e.g. `SUCCESS`/`FAILED`) by the `payments:process-pending` command or the scheduler.

---

3. Initiate a Payout

Creates a new outgoing payment (disbursement) to a beneficiary. The amount is deducted from the merchant's wallet balance.

Endpoint: `POST /api/v1/payouts`

Headers:

Header	Required	Description
`X-API-KEY`	Yes	Merchant's API key
`Content-Type`	Yes	`application/json`

Body Parameters:

Parameter	Type	Required	Description
`amount`	numeric	Yes	Payout amount (e.g. `2000`)
`currency`	string	Yes	Currency code (e.g. `INR`)
`payout_method`	string	Yes	Method used, e.g. `IMPS`, `NEFT`, `RTGS`
`beneficiary_name`	string	Yes	Name of the person/entity receiving funds
`beneficiary_account_number`	string	Yes	Beneficiary's bank account number
`beneficiary_ifsc`	string	Yes	Beneficiary's bank IFSC code
`beneficiary_bank_name`	string	Yes	Name of the beneficiary's bank
`remarks`	string	No	Optional note/reference

cURL Request:

```
curl -X POST http://localhost:8000/api/v1/payouts \
  -H "X-API-KEY: <merchant_api_key>" \
  -H "Content-Type: application/json" \
  -d '{
    "amount": 2000,
    "currency": "INR",
    "payout_method": "IMPS",
    "beneficiary_name": "Priya Verma",
    "beneficiary_account_number": "123456789012",
    "beneficiary_ifsc": "HDFC0001234",
    "beneficiary_bank_name": "HDFC Bank",
    "remarks": "Vendor settlement"
  }'
```

Success Response: Same shape as the Pay-in response — a `transaction_id`, `status: "PENDING"`, and timestamps.

Error Response (insufficient wallet balance) — `422 Unprocessable Entity`:

```
{
  "success": false,
  "message": "Insufficient wallet balance."
}
```

---

4. List / Filter Pay-ins or Payouts

Retrieves a list of the authenticated merchant's transactions, with optional filters.

Endpoint: `GET /api/v1/payins` or `GET /api/v1/payouts`

Query Parameters:

Parameter	Type	Required	Description
`status`	string	No	Filter by status, e.g. `PENDING`, `SUCCESS`, `FAILED`
`date_from`	date (`YYYY-MM-DD`)	No	Only return transactions on/after this date
`date_to`	date (`YYYY-MM-DD`)	No	Only return transactions on/before this date

cURL Request (Pay-ins example):

```
curl "http://localhost:8000/api/v1/payins?status=SUCCESS&date_from=2026-09-
01&date_to=2026-09-02" \
  -H "X-API-KEY: <merchant_api_key>"
```

cURL Request (Payouts example):

```
curl "http://localhost:8000/api/v1/payouts?status=SUCCESS&date_from=2026-09-
01&date_to=2026-09-02" \
  -H "X-API-KEY: <merchant_api_key>"
```

Filters can be combined or omitted — calling the endpoint with no query parameters returns all transactions for the merchant.

---

5. Get a Single Pay-in / Payout by Transaction ID

Fetches full details of one transaction by its `transaction_id`.

Endpoint: `GET /api/v1/payins/{transaction_id}` or `GET /api/v1/payouts/{transaction_id}`

Path Parameters:

Parameter	Type	Required	Description
`transaction_id`	string	Yes	The unique transaction ID returned when the Pay-in/Payout was created

cURL Request (Pay-in example):

```
curl http://localhost:8000/api/v1/payins/PIN20260902AB12CD34EF \
  -H "X-API-KEY: <merchant_api_key>"
```

cURL Request (Payout example):

```
curl http://localhost:8000/api/v1/payouts/<payout_transaction_id> \
  -H "X-API-KEY: <merchant_api_key>"
```

---

6. Get Wallet Balance & Recent Transactions

Returns the merchant's current wallet balance along with a list of recent transactions.

Endpoint: `GET /api/v1/wallet`

Headers:

Header	Required	Description
`X-API-KEY`	Yes	Merchant's API key

cURL Request:

```
curl http://localhost:8000/api/v1/wallet \
  -H "X-API-KEY: <merchant_api_key>"
```

Quick Reference — All Endpoints

Method	Endpoint	Auth Required	Purpose
`POST`	`/api/v1/merchants`	No	Register a new merchant
`POST`	`/api/v1/payins`	Yes	Initiate a Pay-in
`POST`	`/api/v1/payouts`	Yes	Initiate a Payout
`GET`	`/api/v1/payins`	Yes	List/filter Pay-ins

Method	Endpoint	Auth Required	Purpose
GET	/api/v1/payouts	Yes	List/filter Payouts
GET	/api/v1/payins/{transaction_id}	Yes	Get single Pay-in
GET	/api/v1/payouts/{transaction_id}	Yes	Get single Payout
GET	/api/v1/wallet	Yes	Get wallet balance + recent transactions

Notes

- Always replace `<merchant_api_key>` in the examples above with an actual API key obtained from merchant registration or the seeded console output.
- Transactions are created with status `PENDING` and are moved to a final state by the background processing command/scheduler — run `php artisan payments:process-pending` or `php artisan schedule:work` to see status changes reflected.
- Keep `.env` and any API keys out of version control.